The Challenges of Scaling

Training Time ◄

Memory Footprint ◄

Solutions ◄

Energy efficiency ◄

- But : poser les problèmes du DL à grande échelle et des calculs qu'il requiert + survol des solutions (détaillées pendant les 4 jours).
- 4 temps : temps d'entraînement → mémoire → solutions → énergie.
- « Mémoire » = mémoire du GPU, sauf mention contraire.

■ « D'abord : de quel calcul parle-t-on ? Entraînement ou inférence. »

Training / Inference

Training	Inference
– Forward → loss → backward → update	– Forward only → much less memory
– Stores: parameters, gradients, optimizer states, activations	– Model can be compressed: quantization, pruning, distillation…→ Day 4
– Key metrics: throughput (images/s, tokens/s), time to target accuracy	– Key metrics: latency, throughput, memory (+ KV cache for LLMs)

Jean Zay: mostly training (+ evaluation), no model serving
→ DLO-JZ focuses on training

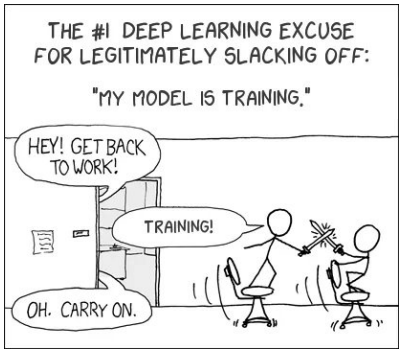
- 2 usages d'un modèle, 2 profils très différents.
- Classic training loop : forward → loss → backward → mise à jour.
* On stocke params, gradients, états de l'optimiseur, activations.
* Ce qui compte : débit (images/s, tokens/s), temps pour atteindre la précision visée.
- Inférence : forward seul → bcp moins de mémoire, modèle compressible (quantization, pruning, distillation, DLO D4).
* Ce qui compte : latence, débit, mémoire (+ KV cache pour les LLM).
- Sur JZ : surtout de l'entraînement (+ évaluation), pas de mise en production → DLO-JZ = entraînement.

■ « À l'entraînement, on se heurte vite à 2 problèmes. »

Constraints of Deep Learning

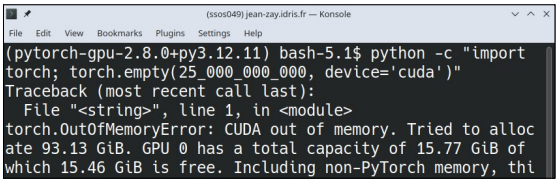
Two problems to deal with

Training Time



Adapted from xkcd #303

GPU memory (Out Of Memory)



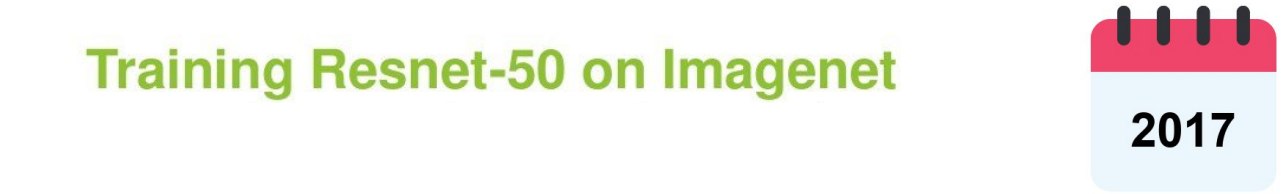


- Temps : de quelques heures à plusieurs semaines
- Mémoire GPU : si ça ne rentre pas → OOM → le job s'arrête.
- Capture réelle sur JZ : ≈ 93 Go demandés sur un GPU de 16 Go → OOM immédiat. C'est le message que vous verrez en TP.
- Les 2 sont liés : ce qui accélère le training (plus gros batch, plus gros modèle) prend souvent plus de mémoire.

■ « Commençons par le temps, avec un exemple classique. » (→ partie Training Time)

Training Time

Training Resnet-50 on Imagenet





Goal:
Classification (1000 classes)
Model :
25 M parameters
Dataset:
1.2 M labeled images

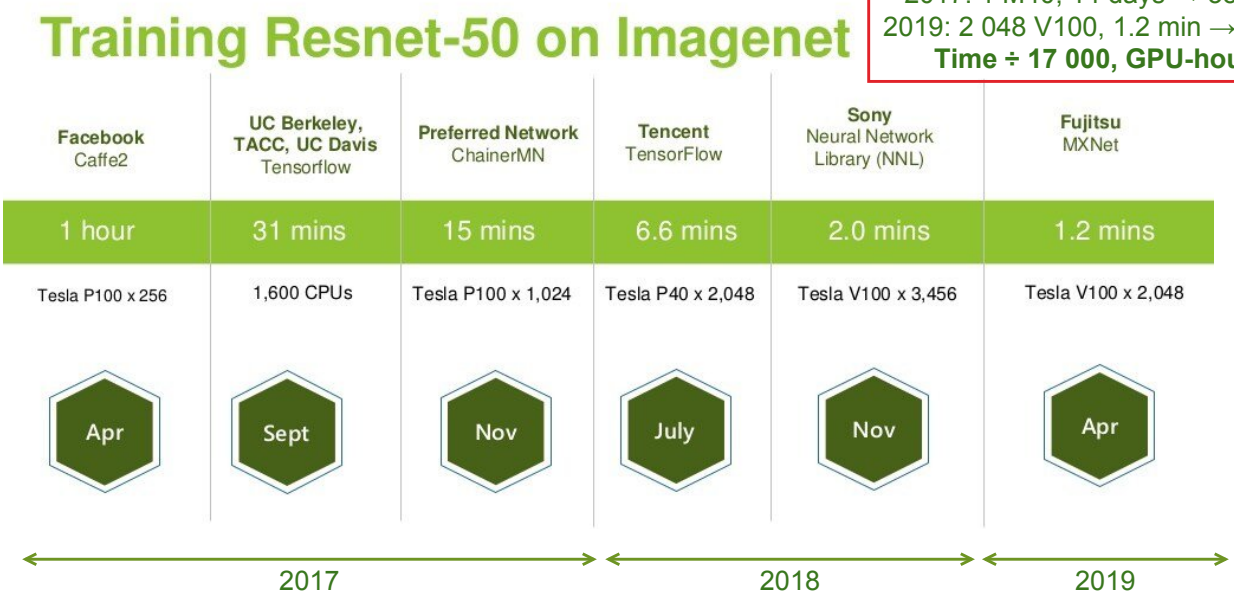
14 Days
1 NVIDIA M40 GPU

- Cas d'école : ResNet-50 sur ImageNet (1 000 classes, 1,2 M images, 25 M params). ImageNet = aussi le fil rouge des TP.
- 2017, 1 GPU (M40) : ≈ 14 jours.
- À ce rythme, impossible de tester beaucoup d'idées.

■ « 14 jours… Voyons comment ce chiffre a fondu en 2 ans. »

Training Time

⚠ **Wall-clock time ≠ GPU-hours**
2017: 1 M40, 14 days → 336 GPU-h
2019: 2 048 V100, 1.2 min → 41 GPU-h
Time ÷ 17 000, GPU-hours ÷ 8



- Course aux records : en 2 ans, 1 h (Facebook, 256 GPU) → 1,2 min (Fujitsu, 2 048 V100)
- Recette : bcp de GPU en parallèle + très grands batchs + réglages d'optim adaptés + GPU plus rapides.
- ■ Temps ≠ GPU-heures :
* slide précédent : 14 j × 1 GPU = 336 GPU-h ;
* 1,2 min × 2 048 GPU ≈ 41 GPU-h.
- Temps ÷ 17 000, mais GPU-h ÷ 8 seulement.
* Le temps baisse surtout grâce au nombre de GPU.
* Les GPU-h baissent grâce à des GPU plus puissants.
- Aller plus vite ≠ consommer moins. Ces 2 unités servent toute la formation.

■ « Ces records mobilisaient des milliers de GPU. fast.ai a montré qu'on pouvait aller très vite avec bien moins. »

Fast.ai tips and engineering

2017 — 14 Days (on 1 GPU) → 2018

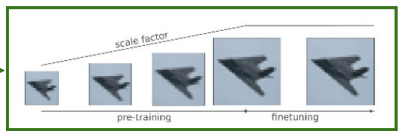
« Now anyone can train Imagenet in 18 Minutes »

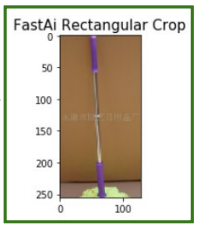
fast.ai - 2018/08/10

From fast.ai & Defense Innovation Unit (DIU) team

- 16 AWS instances × 8 V100 = 128 GPU × 18 min ~ 38 GPU-h => **~40 $**
- Combination of techniques :
  - progressive resizing (fast.ai)
  - rectangular image validation (fast.ai)
  - NCCL/all-reduce (NVIDIA) → Day 2
  - weight decay tuning (Tencent) → Day 3
  - dynamic batch sizes (inspired by Google Brain) → Day 3
  - gradual learning rate warm-up → Day 3

  (Goyal et al 2018, and Leslie Smith 2018)





- 2018 : fast.ai (orga non lucrative, démocratiser IA et DL) + DIU (orga in dept of defense US)

→ ImageNet en 18 min sur 128 V100 loués sur AWS, ≈ 38 GPU-h, ≈ 40 dollars.

- Message : pas besoin d'une infra de géant, les bonnes pratiques comptent autant que le matériel.

- Astuces :

* moins de calcul : petites images au début puis on agrandit ; validation sur images rectangulaires ;

* bonne communication entre GPU : NCCL all-reduce (→ J2) ;

* réglages pour grands batchs : warm-up du LR, weight decay, batch qui grandit (→ J3).

■ « Ça, c'est pour la computer vision. Avec les LLM, on change d'échelle, et un exemple s'est passé ici, sur Jean Zay. »

Training Time: BLOOM

2022

a BigScience initiative

BLOOM

176B params 59 languages Open-access

Goal:

Text Generation Foundation Model (LLM)

Model :

176 B parameters

Dataset:

366 B tokens

117 Days

384 A100 GPUs



1.08 M GPU-hours

~ 4×10^{23} FLOP

- BLOOM (projet BigScience), LLM entraîné sur Jean Zay.
- 176 Md params, 366 Md tokens, 46 langues + 13 langages de programmation.
- Projet ouvert : modèle, données, logs publics.
- “1000 researchers across more than 70 countries to challenge the dominance of closed, proprietary models”
- 384 A100 pendant ≈ 117 j ≈ 1,08 M GPU-h ≈ 4 x 10^{23} FLOP
- 10^23, chiffre énorme: + de 10 000 fois tous les grains de sable de la planète Terre
- si toute l’humanité s’y mettait à raison d’une opé/s par humain => 400 000 ans (1 humain = 230 000x Age univ)

■ « Deux ans plus tard, un modèle de pointe chez Meta. »

Training Time: Llama 3.1



Llama 3.1 405B



Goal:

Text Generation Foundation Model (LLM)

Model :

405 B parameters

Dataset:

15 T tokens

72 Days (est.)

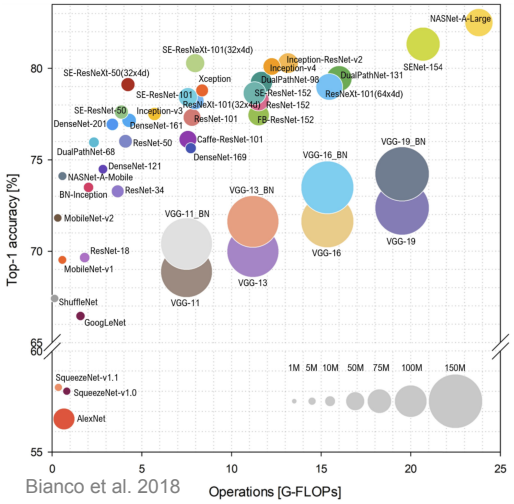
up to 16 384 H100 GPUs

30.8 M GPU-hours
3.8×10^{25} FLOP

- 405 Md params, ≈ 15 T tokens, jusqu'à 16 384 H100, ≈ 31 M GPU-h, 3.8×10^{25} FLOP.
- Durée non publiée par Meta → 72 j = estimation (≈ 2,5 mois).
- vs BLOOM :
- ×100 en calcul ;
- ×30 en GPU-h (H100 ≈ 3× plus rapide qu'A100) ;
- ×40 en nb de GPU → durée du même ordre (quelques mois).
- Ordre de grandeur : sur les 1 456 H100 de JZ → ≈ 2,5 ans.
- ■ *« Pourquoi des modèles toujours plus gros ? Parce que, jusqu'à un certain point, plus gros = meilleur. »*

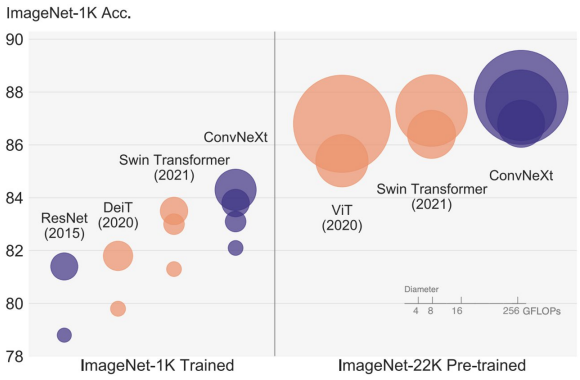
Large Model (Vision)

Vision models on ImageNet (bubble size = nb of params)



Bianco et al. 2018

Effect of training set size (bubble size = FLOPs)



ConvNeXt (Liu et al. 2022)

Bigger models → better accuracy (with enough data)… but architecture matters: VGG-16 (138M params) < ResNet-50 (25M)

- Gauche :

* x = calcul par image, y = précision ImageNet, bulles = nb de params.

* Tendance : plus gros → meilleure précision…

* mais pas que : VGG-16 (138 M params) < ResNet-50 (25 M) → l'architecture compte

* VGG = pré-resnet, modèle large couche, peu profond

- Droite :

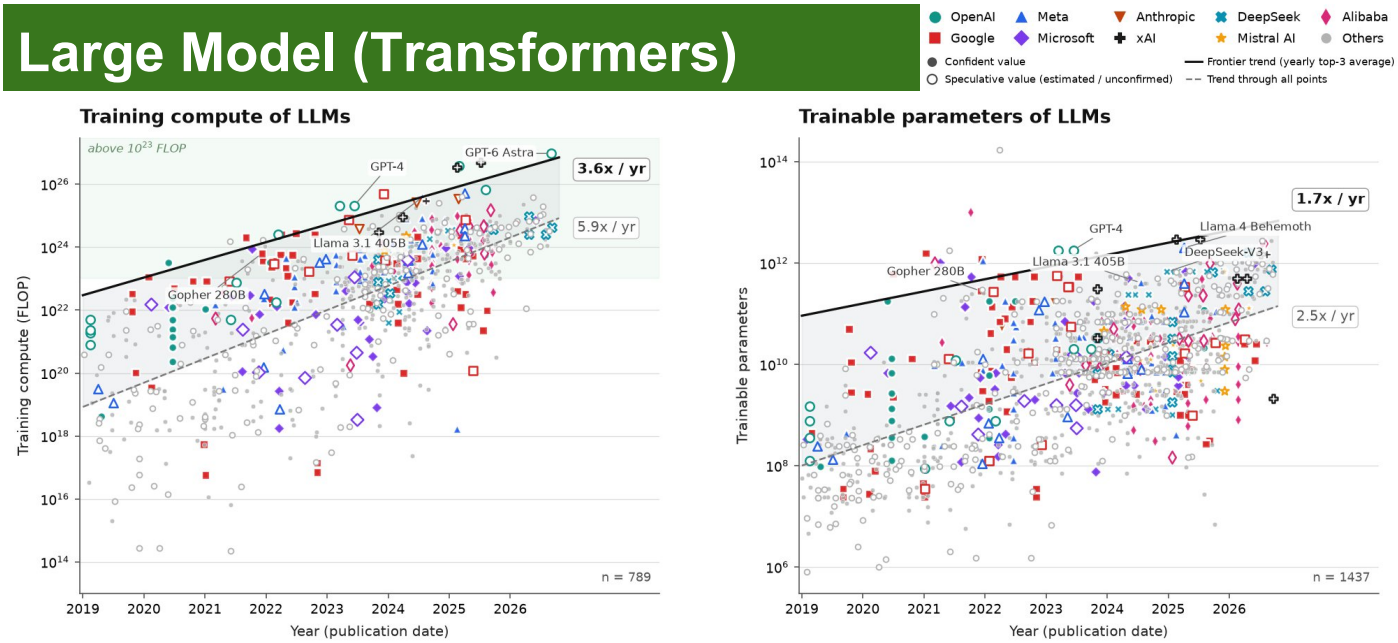
* bulles = calcul (GFLOPs)

* effet d'un entraînement sur ImageNet-1k vs 22k (≈ 14 M images)

* → les gros modèles gagnent plus → il faut assez de données.

■ « Et cette course à la taille s'accélère, dans tous les domaines. »

Large Model (Transformers)



Huge models → long training times and large memory footprint:
1 B parameters = 4 GB in FP32 (2 GB in BF16)
…weights only!

Data from epoch.ai

- Données Epoch AI, tous les LLMs depuis 2019 : 1 point = 1 modèle ; point creux = valeur estimée.
- Gauche : calcul d'entraînement ≈ ×5/an. (les deux lignes : fit tous les modèles vs frontier models) ;

les plus récents > 1026

- Droite : nb de params ≈ ×2/an
- Tailles de plus en plus souvent non publiées + modèles MoE (seule une partie des params sert pour chaque token)

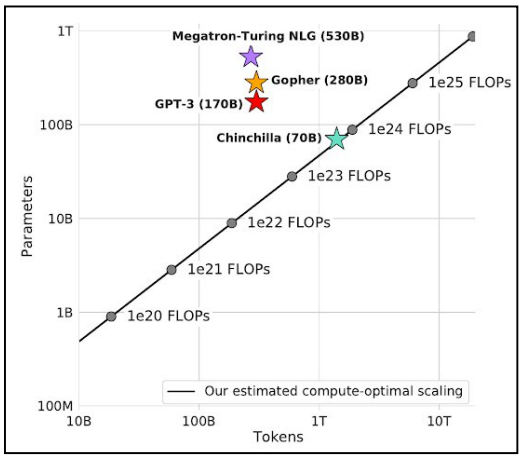
→ le calcul est une meilleure mesure que le nb de params.

- Mémoire : 1 Md params = 4 Go en FP32, 2 Go en BF16… rien que les poids.

■« Mais un modèle plus gros demande aussi plus de données… c'est la loi de Chinchilla. »

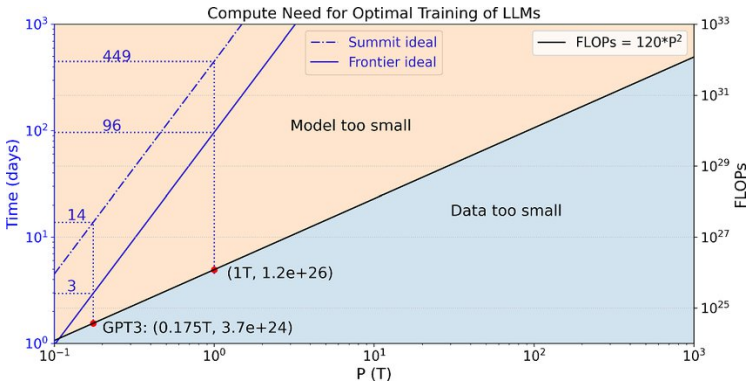
Large Model (Transformers)

Chinchilla Law



$N_{processed\ tokens} \sim 20 \times N_{params}$

Impact



Common approx. for a dense Transformer:
computing cost ~ $6 \times N_{params} \times N_{processed\ tokens}$

Computing cost ~ $120 \times N_{params}^2$
→ 10× more parameters = 100× more compute

- Question : budget de calcul fixé → quelle taille de modèle permise optimale et combien de données nécessaires ?
- Chinchilla (DeepMind, 2022) : optimal ≈ 20 tokens par param.
- GPT-3, Gopher, MT-NLG : trop gros pour les données vues, y a eu une epoque de disette de data
- À coté de ça, approx commune pour un dense transformer : Coût ≈ 6 × N_{params} × N_{tokens} FLOP.
- Vérif Llama 3.1 SLIDE 8 : 6 × 405 Md × 15 T ≈ 4 × 10^{25} → cohérent
- En combinant avec 20 tokens/param : ≈ 120 × N_{params}^2 → ×10 params = ×100 calcul.
- Graphe de droite : temps d'entrainement en jours versus nb de téra-paramètres, dans différent régime
- 1 T params « optimal » → ≈ 10^{26} FLOP → ≈ 100 j sur tout un supercalculateur exascale au pic théorique.
- En pratique on dépasse souvent 20 tokens/param : plus petit modèle = inférence moins chère.
- ■ « Récapitulons : de quoi dépend le temps d'entraînement ? »

Training compute and wall-clock time

time ≈ Compute needed (FLOP) ÷ Compute speed (FLOP/s)

Compute needed grows with:
- model size (parameters, depth)
- data seen = dataset size × number of epochs
- input size: image resolution, sequence length (attention ∝ L^2)

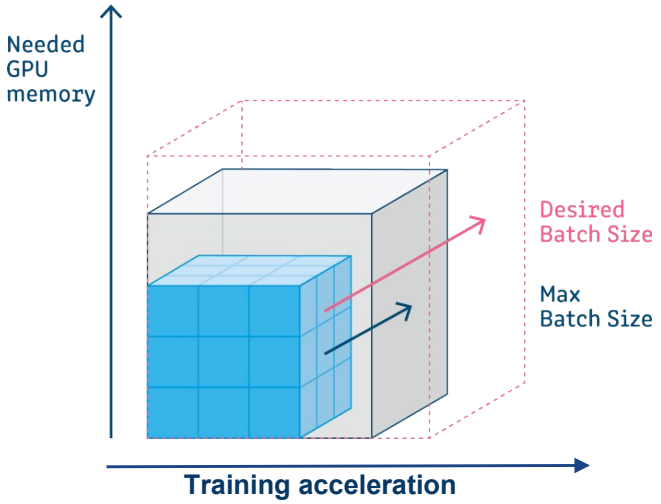
Compute speed = nb of GPUs × peak FLOP/s per GPU × efficiency
(efficiency < 100 %: data loading, I/O, communications, syncs…)

How to speed up? 1) Less compute 2) Better GPUs
3) More GPUs 4) Better GPU usage ← DLO-JZ

- Formule clé : temps ≈ calcul nécessaire (FLOP) ÷ vitesse de calcul (FLOP/s).
- Vocabulaire : FLOP = quantité de calcul ; FLOP/s = vitesse (perf d'un GPU).
- Le calcul nécessaire augmente avec :
- la taille du modèle ;
- les données vues (taille du dataset × nb d'epochs) ;
- la taille des entrées (résolution, longueur de séquence ; attention ∝ L^2).
- Vitesse réelle < pic du GPU : chargement des données, I/O, communications, synchros…
- 4 leviers. DLO-JZ : surtout 4) mieux utiliser les GPU, et 3) bien utiliser plusieurs GPU (J2, J4).

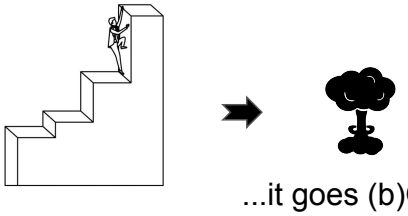
■ « Levier le plus simple pour mieux remplir le GPU : augmenter le batch… et là, on tombe sur le 2e problème. » (→ partie Memory Footprint)

Batch Size & Memory Usage



Bigger batch
- GPU better filled
- more images/s (and fewer steps/epoch)

But memory grows
with the batch, until...



...it goes (b)OOM

Effect on convergence: not free → Day 3

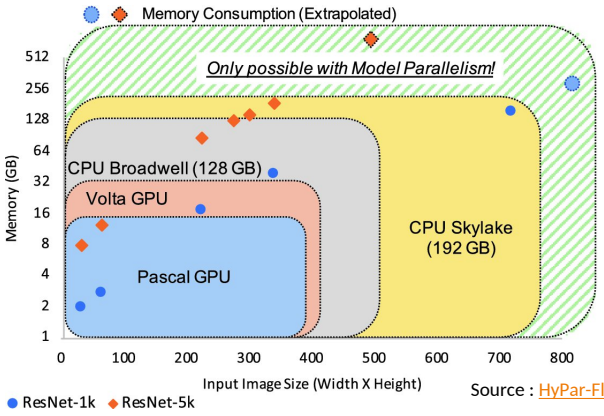
- Batch ↑ → GPU mieux rempli → plus d'images/s, moins d'itérations par epoch.
- Mais la mémoire augmente avec le batch → au-delà du max : OOM.
- De plus, côté convergence, ça n'est pas gratuit (LR à adapter…) → J3.

■ « Le batch n'est pas seul en cause : la taille des entrées compte aussi. »

Large inputs / long sequences

Large input data causes **serious memory occupancy problems** during training, **accentuated by the depth of the model**.

- ➔ Text (N, 100, 500) ***~x1***
- ➔ Image 2D (N, 224, 224, 3) ***~x3***
- ➔ Image 3D (N, 224, 224, 100, 3) ***~x300***
- ➔ Video (N, 100, 224, 224, 3) ***~x300***
- ➔ LLM, long context (N, 128k, 4096) ***~x10 000***



Source : HyPar-Flow

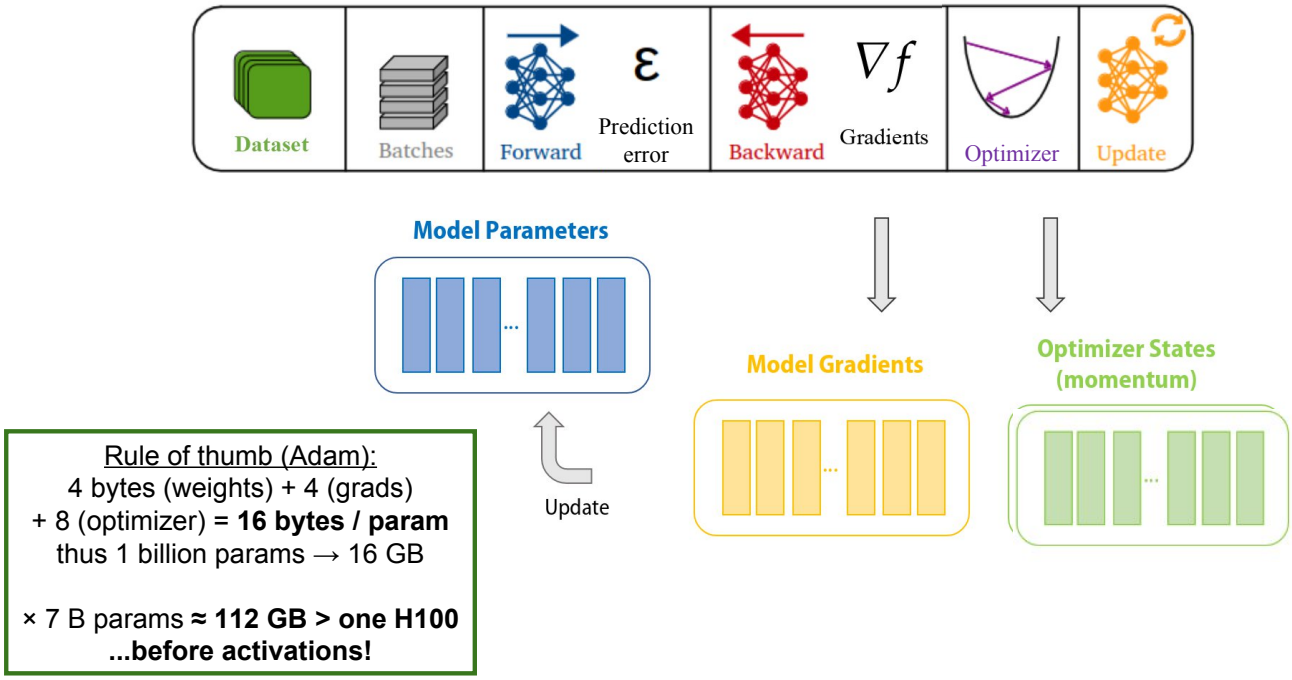
- Ordres de grandeur par échantillon pour different type d'entrées :
- image 2D ≈ ×3 un petit texte ;
- image 3D ou vidéo ≈ ×300 ;
- LLM à long contexte ≈ ×10 000.
- Graphe : memoire occupée par des modèles Resnet versus taille d'image en input
- ResNet sont très profonds (1 000 / 5 000 couches) : La mémoire explose avec la taille d'image

→ dépasse 1 GPU, puis même 1 nœud CPU → il faut découper le modèle.

- Aggravé par la profondeur : plus de couches = plus d'activations à garder

■ « D'où vient toute cette mémoire à stocker ? Regardons ce qu'on stocke, en commençant par le modèle. »

Model Memory Occupancy



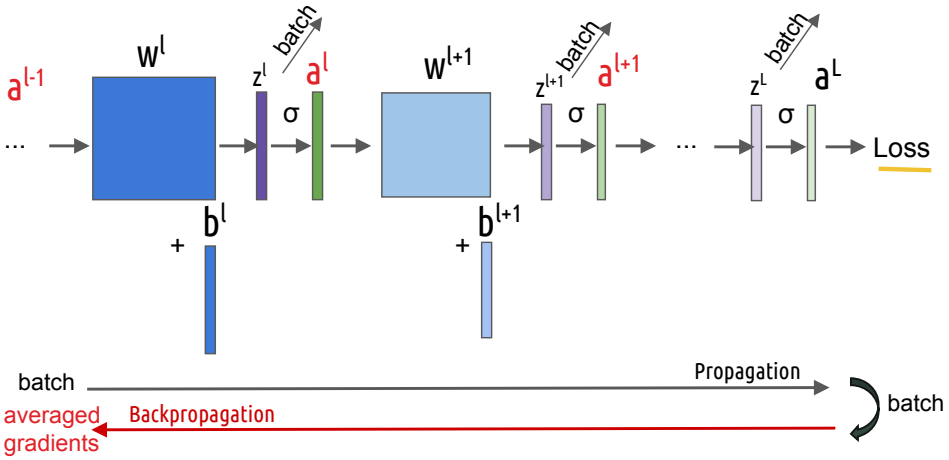
Rule of thumb (Adam):
4 bytes (weights) + 4 (grads)
+ 8 (optimizer) = **16 bytes / param**
thus 1 billion params → 16 GB

× 7 B params **≈ 112 GB > one H100**
...before activations!

- Rappel du training cycle : batch → forward → erreur → backward (gradients) → optimiseur → mise à jour weights
- Mémoire liée au modèle : poids + gradients (même taille) + états de l'optimiseur (Adam : 2 valeurs par param).
- Règle simple avec Adam : ≈ 16 octets/param → 1 Md params ≈ 16 Go ; 7 Md ≈ 112 Go > 1 H100… avant les activations.
- Ne dépend pas de la taille du batch. Ça, c'est la partie fixe.

■ « Reste celle qui grossit avec le batch : les activations. »

Activations Memory Occupancy



Note: For backpropagation, it is necessary to save intermediate activations.
Activation memory ∝ batch size × input size × depth

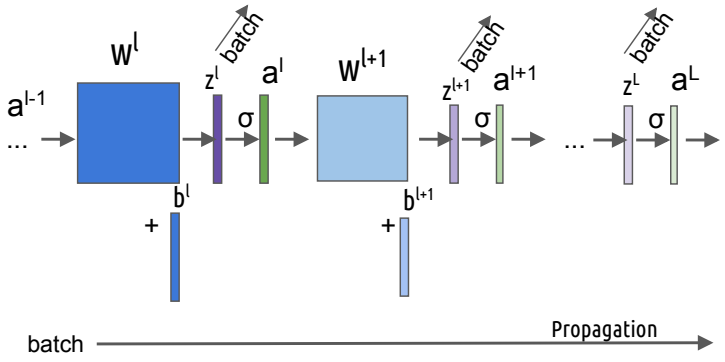
- Pour calculer les gradients au backward, il faut les activations de chaque couche, calculées au forward.

→ On les garde toutes en mémoire jusqu'au backward (sinon : tout recalculer).

- Mémoire des activations /propto batch × taille des entrées × profondeur.
- Souvent la plus grosse part de la mémoire à l'entraînement (sauf très gros modèles).

■ « Et quand il n'y a pas de backward, comme en validation ? »

Inference & evaluation



Propagation

$$a^l = \sigma(w^l a^{l-1} + b^l) = \sigma(z^l)$$

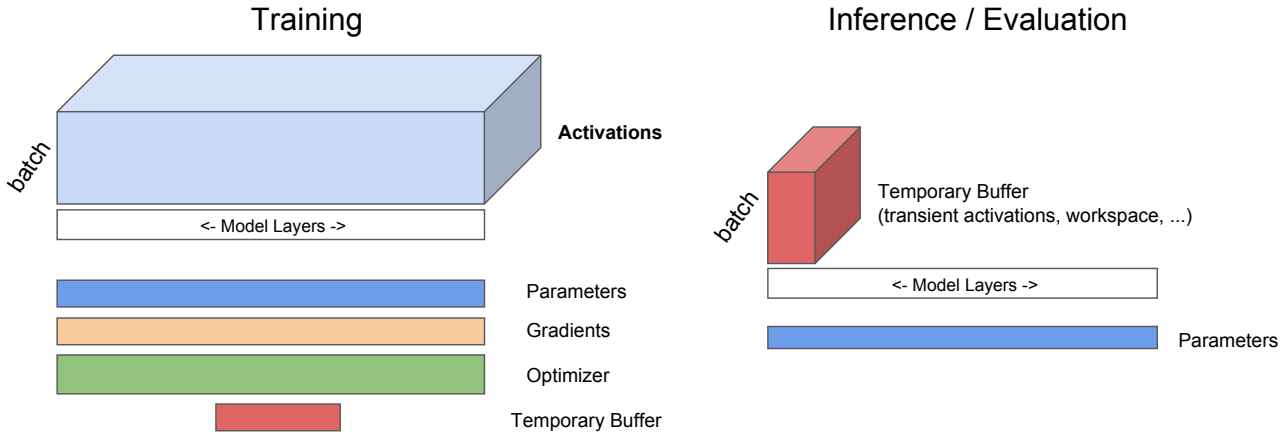
```
…
model.eval()                    # dropout / BatchNorm
                                  in eval mode
with torch.inference_mode():    # no autograd graph →
                                  activations not kept
    val_outputs = model(val_images)
    loss = criterion(val_outputs, val_labels)
model.train()                   # back to training mode
…
```

No backward
- ➢ no stored activations
- ➢ much less memory
- ➢ (bigger validation batch possible)

- Pas de backward → pas besoin de garder les activations : chaque couche libère la précédente.
- PyTorch :
* `model.eval()` → dropout / BatchNorm en mode éval ;
* `torch.inference_mode()` (ou `no_grad()`) → pas de graphe autograd → bcp moins de mémoire, un peu plus rapide.
- En pratique : un batch de validation plus gros est du coup souvent possible.
- Penser à `model.train()` après.

■ « Résumons l'empreinte mémoire sur un schéma. »

Memory Footprint

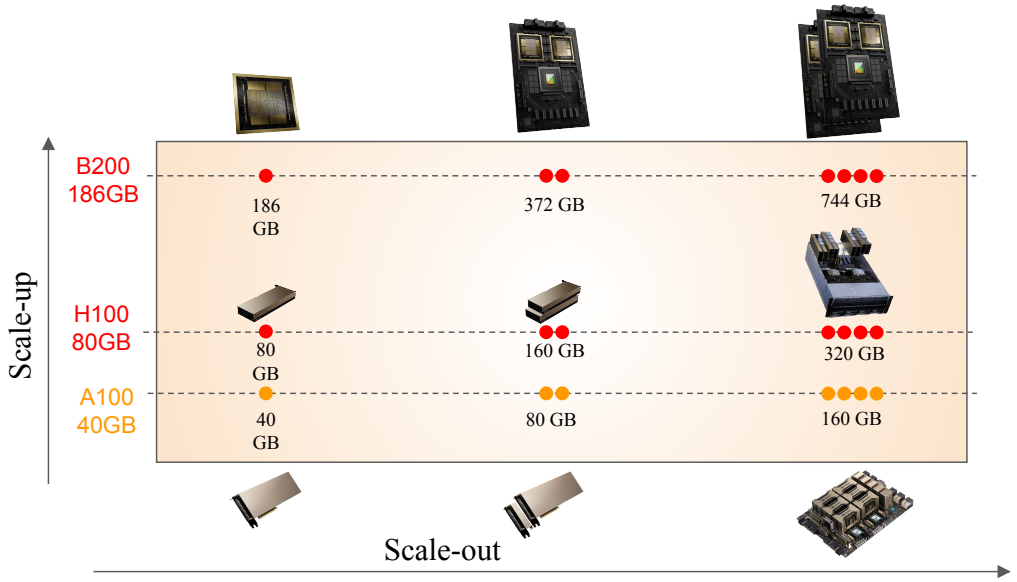


Small models: activations dominate
Huge models: parameters + gradients + optimizer alone can exceed GPU memory

- Training : activations (grossissent avec batch et profondeur) + params + gradients + optimiseur + buffers temporaires.
- Inférence : params + petit buffer.
- Petit modèle : activations dominantes → on joue sur le batch.
- Très gros modèle : params + gradients + optimiseur peuvent déjà dépasser 1 GPU → à répartir (FSDP, J4).

■ « On a vu les 2 problèmes de l'entraînement de gros modèles, le temps et la mémoire. Passons aux solutions, d'abord côté matériel. » (→ partie Solutions)

System Solutions



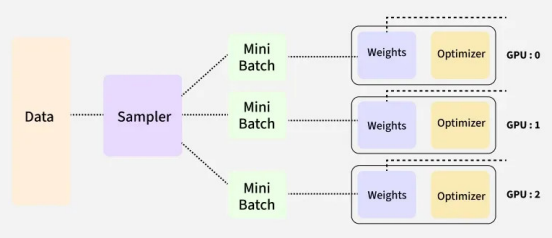
Scale-up = better GPUs ; Scale-out = more GPUs

- côté matériel, 2 axes à explorer
- Scale-up : GPU plus récent → plus de mémoire, plus rapide (A100 → H100 → B200).
- Petite note: sur JZ, A100 et H100 ont 80 Go.
- Scale-out : plus de GPU → mémoire et calcul s'additionnent… si on sait répartir le travail.

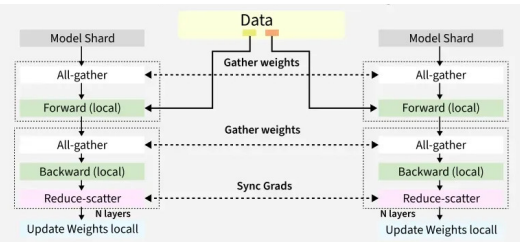
■« Plus de GPU, d'accord… mais comment leur répartir le travail ? »

Distribution / Scale-out

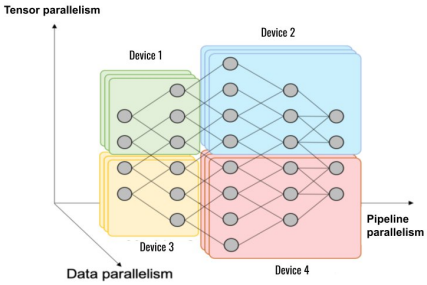
Distributed Data Parallelism (DDP) - Day 2



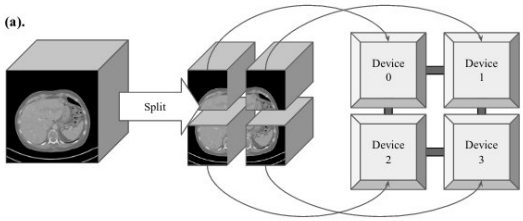
Sharded data parallelism (FSDP) - Day 4



Model Parallelism (Tensor + Pipeline) - Day 4



Spatial/domain parallelism



- 4 façons de répartir le travail :

* DDP : 1 copie du modèle par GPU, chacun traite une part du batch, gradients moyennés → accélère, mais même mémoire par GPU (J2). Ça permet aussi de gros batches. Le modèle doit tenir en mémoire !

* Sharded DP (FSDP) : batch pareil, mais poids, gradients et états de l'optimiseur découpés entre GPU → gros gain mémoire (J4).

* Model parallelism : on découpe le modèle, par couches (pipeline) ou à l'intérieur des couches (tensor). Pour les très gros modèles (J4).

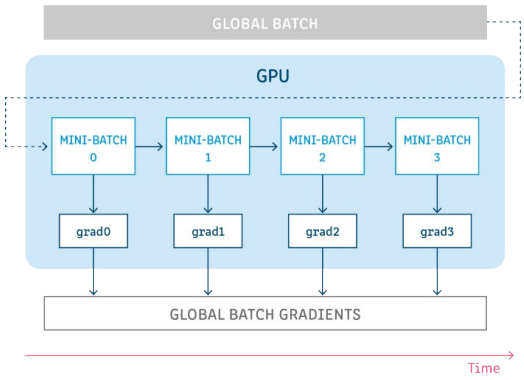
* Spatial : on découpe la donnée elle-même (ex. image 3D médicale). Rare, pas abordé.

- LLM : on combine souvent plusieurs approches.

■ « Et sans GPU en plus ? 2 astuces pour économiser la mémoire. »

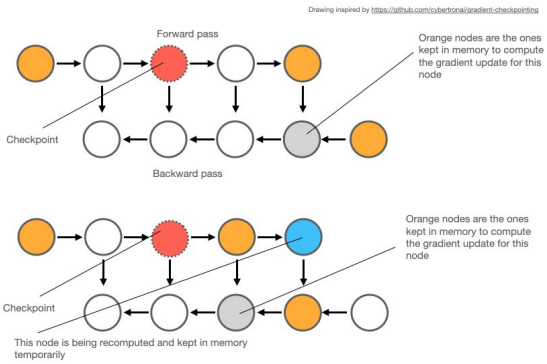
Workaround Solutions

Gradient accumulation



Large effective batch with little memory but sequential (no speed-up)

Gradient / Activation checkpointing



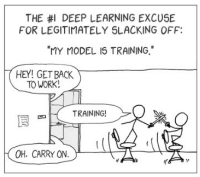
Keep only some activations, recompute the others in backward
≈ +30 % compute

- 2 astuces sur 1 GPU : moins de mémoire, mais pas plus rapide.
- Gradient accumulation : plusieurs petits batchs à la suite, gradients cumulés, 1 seule mise à jour → comme un gros batch, mais séquentiel.
- Activation checkpointing : on ne garde que quelques activations, on recalcule les autres au backward → mémoire ↓↓, calcul ↑ (≈ +30 %).
- Souvent combinées avec la distribution.
- - ■ *« Temps, mémoire… il reste un 3e problème. »* (→ partie Energy)

A 3rd problem to deal with ...

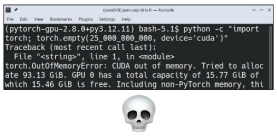
Two problems to deal with

Training Time



Adapted from xkcd #303

GPU memory (Out Of Memory)



Energy consumption



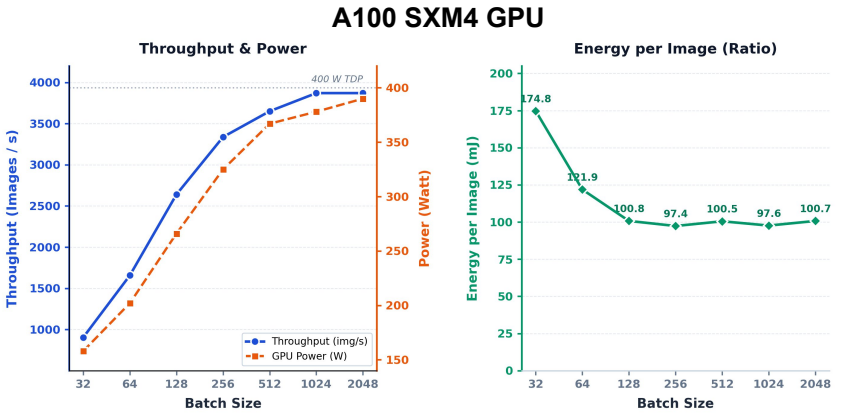
- En plus du temps et de la mémoire : l'énergie.
- Coût, empreinte carbone, limites physiques des centres de calcul (alimentation, refroidissement).
- Bonne nouvelle : optimiser son code = gagner du temps ET de l'énergie.

■ « Combien consomme un GPU, et quand est-il le plus efficace ? »

Power draw & energy efficiency

	H100 PCIe	H100 SXM5	A100 PCIe	A100 SXM4	V100 PCIe	V100 SXM2
Max Power	350W	700W	250W	400W	250W	300W
Idle Power	~30W	~55W	~30W	~60W	~40W	~45W
Perf. (vs V100 SXM2)	x3.5	x4	x1.75	x2	x0.9	x1

For a node:
The CPU (often 2 processors) consumes what approximately 1 GPU consumes.



- Bigger batch → more watts… but even more images/s
- Batch 32 → 1024: ≈ 45 % less energy per image
- A fully used GPU is the most energy-efficient

- Tableau des GPU de JZ:
* puissance max, jusqu'à 700 W (H100 SXM) ;
* au repos, un GPU consomme encore 30-60 W ;
* GPU récents plus efficaces : H100 SXM = ×4 de perf pour ×2,3 de puissance (vs V100).
- Par nœud : les CPU consomment à peu près comme 1 GPU.
- Graphe (1 GPU, on augmente le batch) :
* débit et puissance ↑, mais le débit monte plus vite → énergie par image ↓ (≈ –45 % entre batch 32 et 1024) ;
* l'essentiel du gain est acquis vers 256, ensuite plateau.
→ GPU bien rempli = plus efficace ; GPU qui attend = gaspillage.
■ « Conclusion pratique : ce qu'il faut surveiller, ce sont les GPU-heures. »

Energy Consumption / GPU-hours

Energy Saving
≅
GPU-hours Saving

System optimization (DLO-JZ)

- Maximize throughput (without losing accuracy)
- Remove GPU idle time (e.g. waiting for data)
- Use the right number of GPUs: neither too many nor too few

- Pour un type de GPU : énergie ≈ GPU-heures × puissance moyenne

→ économiser des GPU-h = économiser de l'énergie (et des heures sur son projet).

- 3 leviers « système » vus pendant la semaine DLO-JZ :

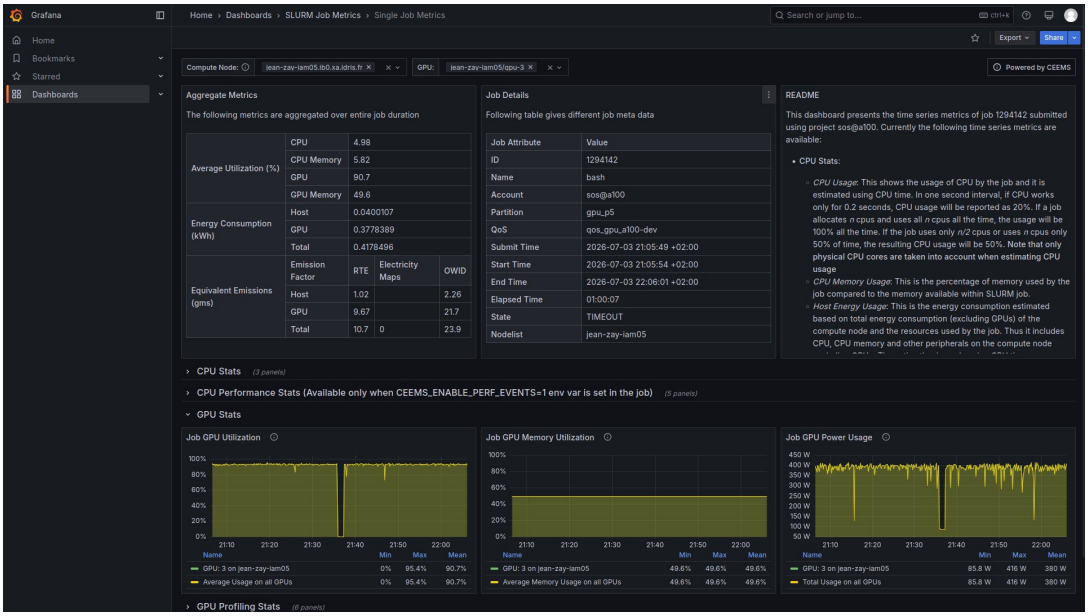
* débit max sans perdre en précision ;

* supprimer les temps morts (attente des données) ;

* choisir le bon nombre de GPU.

Energy Consumption / GPU-hours

Introducing: CEEMS



- J’en profite pour mentioner un outil disponible sur Jean Zay, CEEMS, qui peut vous aider
- Stats CPU, GPU, mem, IO, et donc conso énergétique ; stats agrégées et temporelles

Energy Consumption / GPU-hours

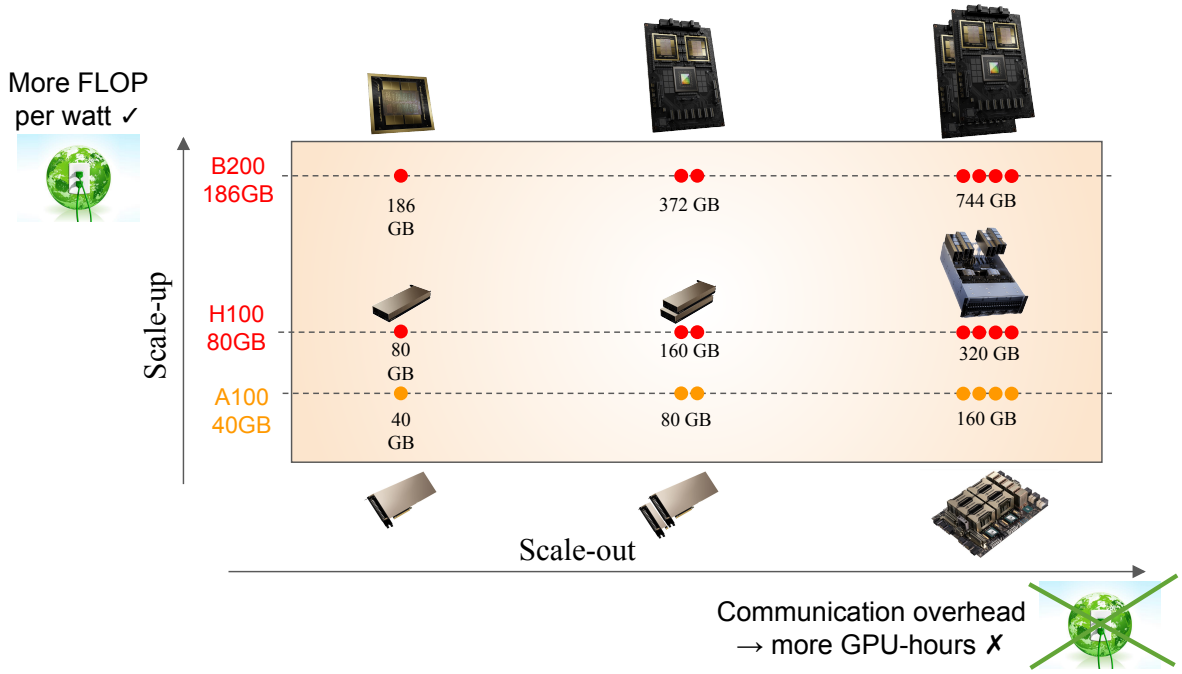
Energy Saving
≅
GPU-hours Saving

System optimization (DLO-JZ)

- Maximize throughput (without losing accuracy)
- Remove GPU idle time (e.g. waiting for data)
- Use the right number of GPUs: neither too many nor too few

■ « Reprenons maintenant le schéma scale-up / scale-out, côté énergie. »

Energy Consumption / GPU-hours



- Scale-up (GPU récents) : plus de calcul par watt

→ bon pour l'énergie.

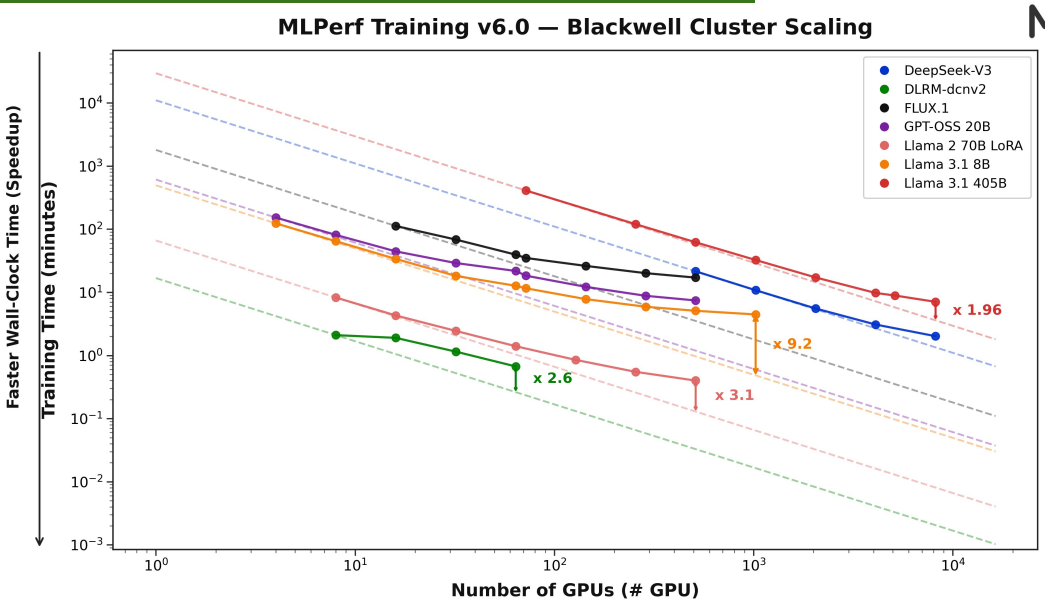
- Scale-out : plus rapide, mais communications en plus

→ souvent plus de GPU-h au total pour le même entraînement.

→ ajoutez des GPU seulement tant que l'efficacité reste bonne.

■ « Jusqu'où ajouter des GPU sans gaspiller ? Début de réponse avec MLPerf. »

ML Perf Result - Scaling



Dashed = ideal scaling · Solid = measured · Gap = wasted GPU-hours
(best case: vendor-optimized code)

- MLPerf : benchmark de référence = temps pour atteindre une qualité cible.
- Ici : résultats de juin 2026 (v6.0), GPU Blackwell.
- Pointillés = idéal (2× plus de GPU → 2× plus rapide). Traits pleins = mesuré. Écart = GPU-h gaspillées.
- Gros modèles (Llama 3.1 405B, DeepSeek-V3) : proches de l'idéal jusqu'à des milliers de GPU.
- Plus petits (DLRM, Llama 3.1 8B, FLUX…) : saturent dès quelques dizaines de GPU.
- Codes ultra-optimisés par les constructeurs = meilleur cas.

ML Perf Result - Scaling

Optimizations fall into four major categories:

A. Low-Precision Numerics & 2nd-Gen Transformer Engine
- FP8 Mixed-Precision Training (E4M3 & E5M2)
- Block-Level Dynamic Scaling
- Tensor Memory Accelerator (TMA) & Asynchronous Warp Specialization

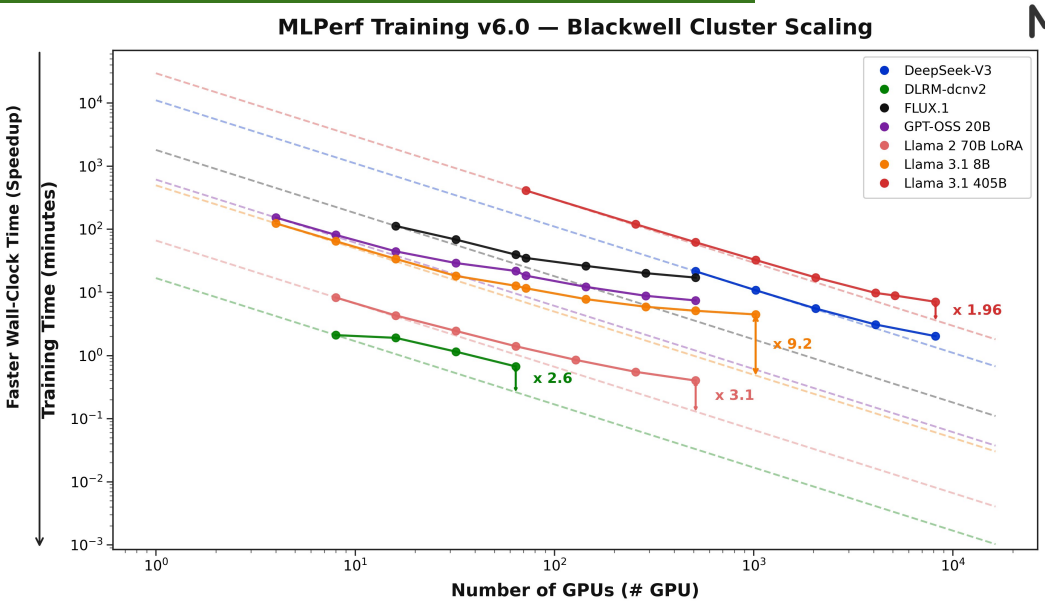
B. 5D Distributed Parallelism & Hybrid Schedules
1. NVLink-5 Domain Tensor Parallelism (TP):
2. Zero-Bubble Pipeline Parallelism (ZB-PP):
3. Dual-Pipe Expert Parallelism (EP) for MoEs:
4. Context Parallelism (CP):
5. Fully Sharded Data Parallelism (ZeRO-3 / FSDP):

C. Model-Specific Algorithmic & Kernel Breakthroughs

D. In-Network Collective Acceleration
- SHARP (Scalable Hierarchical Aggregation and Reduction Protocol)
- CUDA Graphs

- Pour ceux que ça intéressent, la liste des optimisations faites sur les modèles dans le cadre de MLPerf

ML Perf Result - Scaling



Dashed = ideal scaling · Solid = measured · Gap = wasted GPU-hours
(best case: vendor-optimized code)

Code « normal » → saturation plus tôt → mesurer son propre scaling (TP J2).

■ « Dernier levier, souvent le plus rentable : éviter les entraînements inutiles. »

Energy : Methodology

Methodology: avoid unnecessary trainings

1. Reuse hyperparameters from publications (reproduce the state of the art)

2. Tune on small models, then scale up

3. Only then: automatic HPO (Day 4)

Ne pas refaire ce qui existe déjà.

- 1) Reprendre les hyperparamètres de la littérature, reproduire l'état de l'art.

- 2) Régler sur des petits modèles, puis passer à l'échelle (certains HP, comme le LR, ne se transfèrent pas tels quels).

- 3) En dernier recours : recherche automatique (HPO, J4), coûteuse.

■ « Transition → maintenant on va rentrer dans le vif du sujet avec la présentaiton par Martin du script principal dont vous allez vous servir pendant la formation »